

Test Markers Reference

This document describes all available pytest markers for organizing and running tests.

Available Markers

Execution Speed

`@pytest.mark.slow`

Marks tests that take more than 1 second to run.

Usage:

```
@pytest.mark.slow
def test_large_repository_operation():
    # Long-running test
    pass
```

Run:

```
# Skip slow tests
pytest -m "not slow"

# Run only slow tests
pytest -m "slow"
```

Test Type

`@pytest.mark.unit`

Marks tests as unit tests (testing single functions/classes in isolation).

Usage:

```
@pytest.mark.unit
def test_add_function():
    assert add(1, 2) == 3
```

Run:

```
pytest -m "unit"
```

`@pytest.mark.integration`

Marks tests as integration tests (testing interaction between components).

Usage:

```
@pytest.mark.integration
def test_git_engine_with_state_manager():
    # Tests interaction between components
    pass
```

Run:

```
pytest -m "integration"
```

@pytest.mark.e2e

Marks tests as end-to-end tests (testing complete workflows).

Usage:

```
@pytest.mark.e2e
def test_complete_workpad_lifecycle():
    # Create -> Edit -> Test -> Promote
    pass
```

Run:

```
pytest -m "e2e"
```

Interface Type

@pytest.mark.cli

Marks tests specific to the CLI interface.

Usage:

```
@pytest.mark.cli
def test_repo_list_command(cli_runner):
    result = cli_runner.invoke(repo_list)
    assert result.exit_code == 0
```

Run:

```
pytest -m "cli"
```

@pytest.mark.tui

Marks tests specific to the TUI (Text User Interface).

Usage:

```
@pytest.mark.tui
def test_command_palette():
    # Test TUI component
    pass
```

Run:

```
pytest -m "tui"
```

@pytest.mark.gui

Marks tests specific to the GUI (Graphical User Interface).

Usage:

```
@pytest.mark.gui
def test_monaco_editor():
    # Test GUI component
    pass
```

Run:

```
pytest -m "gui"
```

External Dependencies

@pytest.mark.ai

Marks tests that require AI API access.

Usage:

```
@pytest.mark.ai
def test_ai_code_generation():
    # Requires ABACUS_API_KEY or similar
    pass
```

Run:

```
# Skip AI tests (when API keys not configured)
pytest -m "not ai"

# Run only AI tests
pytest -m "ai"
```

Test Priority

`@pytest.mark.smoke`

Marks critical tests that should run in smoke test suite (< 30 seconds total).

Usage:

```
@pytest.mark.smoke
def test_system_starts():
    # Critical startup test
    pass
```

Run:

```
pytest -m "smoke"
```

Async Tests

`@pytest.mark.asyncio`

Marks async tests (handled by pytest-asyncio plugin).

Usage:

```
@pytest.mark.asyncio
async def test_async_operation():
    result = await some_async_function()
    assert result is not None
```

Run:

```
# Async tests run automatically with pytest-asyncio installed
pytest
```

Marker Combinations

You can combine markers for fine-grained test selection:

```
# Run fast unit tests only
pytest -m "unit and not slow"

# Run all integration tests except AI tests
pytest -m "integration and not ai"

# Run smoke tests for CLI
pytest -m "smoke and cli"

# Run all tests except slow and e2e
pytest -m "not slow and not e2e"
```

Automatic Marker Assignment

Some markers are automatically assigned based on test location (see `tests/conftest.py`):

- Tests in `tests/integration/` → `@pytest.mark.integration`
- Tests in `tests/e2e/` → `@pytest.mark.e2e`
- Tests in `tests/cli/` → `@pytest.mark.cli`
- Tests in `tests/ui/tui/` → `@pytest.mark.tui`
- Integration and E2E tests → `@pytest.mark.slow`

CI/CD Usage

Pull Request Checks

```
# Fast feedback (< 30s)
pytest -m "smoke" --maxfail=5
```

Pre-merge Validation

```
# Unit + integration (< 2 minutes)
pytest -m "unit or integration" -m "not slow"
```

Nightly Full Suite

```
# Everything including slow tests
pytest
```

Pre-deployment

```
# Smoke + critical paths
pytest -m "smoke or (e2e and not slow)"
```

Best Practices

1. Mark Your Tests

Always add appropriate markers to new tests:

```
@pytest.mark.unit
@pytest.mark.cli
def test_new_cli_command():
    pass
```

2. Use Multiple Markers

Tests can have multiple markers:

```
@pytest.mark.slow
@pytest.mark.integration
@pytest.mark.ai
def test_ai_integration_workflow():
    pass
```

3. Skip Appropriately

Use markers to skip tests when dependencies unavailable:

```
@pytest.mark.ai
@pytest.mark.skipif(not has_api_key(), reason="API key not configured")
def test_ai_feature():
    pass
```

4. Document Custom Markers

If you add new markers, update:

- `pytest.ini` (register marker)
- `tests/conftest.py` (if auto-assigning)
- This file (document usage)

Marker Summary Table

Marker	Category	Auto-assigned	Typical Duration
<code>slow</code>	Speed	Yes (integration/e2e)	> 1s
<code>unit</code>	Type	No	< 0.1s
<code>integration</code>	Type	Yes (path-based)	0.1-1s
<code>e2e</code>	Type	Yes (path-based)	1-5s
<code>cli</code>	Interface	Yes (path-based)	varies
<code>tui</code>	Interface	Yes (path-based)	varies
<code>gui</code>	Interface	No	varies
<code>ai</code>	Dependency	No	varies
<code>smoke</code>	Priority	No	< 1s
<code>asyncio</code>	Technical	No	varies

Examples

Example 1: Fast Unit Test

```
import pytest

@pytest.mark.unit
def test_calculation():
    """Fast, isolated test."""
    assert calculate(2, 3) == 5
```

Example 2: Slow Integration Test

```
import pytest

@pytest.mark.slow
@pytest.mark.integration
def test_database_migration():
    """Tests component interaction, takes time."""
    db.migrate()
    assert db.schema_version == 5
```

Example 3: Critical Smoke Test

```
import pytest

@pytest.mark.smoke
@pytest.mark.cli
def test_cli_starts():
    """Critical test for CI smoke tests."""
    result = cli_runner.invoke(main, ['--version'])
    assert result.exit_code == 0
```

Example 4: AI-dependent Test

```
import pytest

@pytest.mark.ai
@pytest.mark.integration
@pytest.mark.skipif(not os.getenv('ABACUS_API_KEY'), reason="No API key")
def test_ai_code_generation():
    """Requires external AI service."""
    code = ai_orchestrator.generate("create hello function")
    assert "def hello" in code
```

Updating Markers

When adding a new marker:

1. Register in `pytest.ini` :

```
markers =  
    mymarker: description of marker
```

1. **Document here** with usage examples
 2. **Update** `conftest.py` if auto-assignment needed
 3. **Update CI scripts** to use new marker
-

For more information, see:

- `pytest.ini` - Marker registration
- `tests/conftest.py` - Auto-assignment logic
- `docs/TESTING_GUIDE.md` - Comprehensive testing guide